

20260501 3회차 발표 자료

지난 시간 복습

Policy Evaluation & Policy Improvement

1. Policy Evaluation

Bellman Expectation Equation (expression)

$$V_{\pi}(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_{\pi}(s')]$$

Input: π , P , R , γ , θ (convergence threshold)

Initialize $V(s) = 0$ for all s

repeat:

$\Delta \leftarrow 0$

for each $s \in S$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \cdot \sum_{s'} P(s'|s,a) \cdot [R(s,a,s') + \gamma$
 $\cdot V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

return V

2. Policy Improvement

Greedy Policy Improvement (expression)

$$\pi'(s) = \arg \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_{\pi}(s')]$$

also,

$$\pi'(s) = \arg \max_a Q_\pi(s, a)$$

where,

$$Q_\pi(s, a) = \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_\pi(s')]$$

```

Input: V (from policy evaluation),  $\pi$ , P, R,  $\gamma$ 

policy_stable  $\leftarrow$  true

for each  $s \in S$ :
    old_action  $\leftarrow \pi(s)$ 

     $\pi'(s) \leftarrow \operatorname{argmax}_a \sum_{s'} P(s' | s, a) \cdot [R(s, a, s') + \gamma \cdot V(s')]$ 

    if old_action  $\neq \pi'(s)$ :
        policy_stable  $\leftarrow$  false

return  $\pi'$ , policy_stable

```

3. full

```

Initialize  $\pi$  arbitrarily,  $V(s) = 0$ 

repeat:
    # Step 1: Evaluate current policy
     $V \leftarrow \text{PolicyEvaluation}(\pi)$ 

    # Step 2: Improve policy greedily
     $\pi', \text{stable} \leftarrow \text{PolicyImprovement}(V, \pi)$ 

     $\pi \leftarrow \pi'$ 

until stable == true    #  $\pi = \pi^*$  (optimal policy reached)

```

```
return  $\pi$ ,  $V$ 
```

문제점

우리는 실제 world model을 모른다!! → 확률 분포 P 나, 모든 R 을 알 수 없다. (실제 해보기 전 까진...)

⇒ $V_{\pi}(s)$ 를 "추정" 하면 된다. 실제로 얻은 data trajectory를 통해...

방법 1: MC, Monte Carlo

idea: 하나의 에피소드를 마친 뒤 얻은 실제 G_t 를 평균내어 V^{π} 를 추정하기

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

First Visit

State s 가 에피소드에서 처음 등장한 시점의 G_t 만 사용.

```
for each episode:
    Generate episode:  $s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T$ 

    visited = {}
    for  $t = 0$  to  $T-1$ :
        if  $s_t$  NOT in visited:
            visited.add( $s_t$ )
             $G \leftarrow$  return from  $t$ 
             $N(s_t) += 1$ 
             $V(s_t) += G$ 

     $V(s) = \text{sum\_G}(s) / N(s)$ 
```

특징

- **unbiased** estimator — 이론적으로 깔끔
- 방문 횟수가 적으면 variance 높음

Every-Visit MC

State s 가 등장할 때 마다 G_t 전부 사용.

```
for each episode:
    Generate episode: s0, a0, r1, s1, a1, r2, ..., sT

    for t = 0 to T-1:
        G ← return from t          # 중복 방문도 전부 사용
        N(s_t) += 1
        V(s_t) += G

V(s) = sum_G(s) / N(s)
```

- **biased** (같은 에피소드 내 G들은 독립이 아님) → 하지만 **consistent** (샘플 무한히 많으면 수렴)
- 데이터 효율이 더 좋음 (같은 에피소드에서 더 많이 학습)
- 실제로는 First-Visit보다 자주 쓰임

Incremental MC

위 두 방법은 에피소드를 저장 후 나중에 평균을 내는 batch 방식.

Incremental은 에피소드가 끝날 때마다 즉시 업데이트

$$V(s) \leftarrow V(s) + \frac{1}{N(s)}(G - V(s))$$

also,

더 나아가, $\frac{1}{N(s)}$ 대신 고정 step-size α 를 쓰면:

$$V(s) \leftarrow V(s) + \alpha(G - V(s))$$

```
for each episode:
    Generate episode, compute G for each visited s

    for each (s, G) in episode:
        N(s) += 1
        V(s) ← V(s) + (1/N(s)) * (G - V(s))
```

방법 2: TD(0), Temporal Difference

MC의 한계에서 출발

MC는 에피소드가 끝나야 G_t 를 계산할 수 있다.

TD의 핵심 아이디어: 실제 return G_t 대신, 현재 추정값 $V(s')$ 로 target을 만든다.

```
MC:  $V(s) \leftarrow V(s) + \alpha(G_t - V(s))$  #  $G_t$ 는 에피소드 끝나야  
알 수 있음  
TD:  $V(s) \leftarrow V(s) + \alpha(R + \gamma V(s') - V(s))$  # 한 스텝만으로 즉시  
업데이트, bootstrapping
```

$$V(s_t) \leftarrow V(s_t) + \alpha \underbrace{[R_{t+1} + \gamma V(s_{t+1}) - V(s_t)]}_{\delta_t : \text{TD error}}$$

- $R_{t+1} + \gamma V(s_{t+1}) \rightarrow$ **TD target** (bootstrap)
- $\delta_t \rightarrow$ **TD error** : 예측과 실제의 차이

```
Initialize  $V(s) = 0$  for all  $s$ 
```

```
for each episode:
```

```
     $s \leftarrow$  initial state
```

```
    for each step:
```

```
         $a \leftarrow \pi(s)$ 
```

```
         $s', r \leftarrow \text{env.step}(a)$ 
```

```
         $\delta \leftarrow r + \gamma * V(s') - V(s)$  # TD error
```

```
         $V(s) \leftarrow V(s) + \alpha * \delta$  # 즉시 업데이트
```

```
         $s \leftarrow s'$ 
```

```
        if  $s$  is terminal: break
```

에피소드가 끝나기 전에도 계속 학습 $\rightarrow$ online learning 가능

MC vs TD vs DP 비교

	MC	TD	DP
Bootstrapping	✗	✓	✓
Sampling	✓	✓	✗
에피소드 완료 필요	✓	✗	✗
Model 필요	✗	✗	✓
Bias	Low	High	-
Variance	High	Low	-

지난 내용 overview

DP

└ Model 알아야 함, bootstrap 0

MC

└ Model 불필요, 에피소드 끝까지 필요
└ unbiased, high variance

TD(0)

└ Model 불필요, 한 스텝만으로 업데이트
└ biased, low variance
└ online 가능

!!오늘 내용!!

↓

Q-Learning / SARSA (TD를 control에 적용)

↓

DQN (Q-Learning + Neural Network)

지금까지 배운 것이, Generalized Policy Improvement 이다.

환경 모델(Dynamics, Reward)을 모를 때, **경험만으로 $Q(s, a)$ 를 평가하고 정책을 향상**시키는 순환 과정.

이 과정에서 현재 아는 최고의 보상을 선택하는 활용(Exploitation)과 더 나은 경로를 찾기 위한 새로운 시도인 탐험(Exploration) 사이의 딜레마가 발생.

무조건 현재 최대 가치를 선택하는 대신, 일정 확률 ϵ 으로 무작위 행동을 취해 다양한 상태를 경험하게 만들.

→ **ϵ -greedy**

$$\pi(a|s) = \begin{cases} \arg \max_a Q(s, a) & \text{prob} = 1 - \epsilon + \frac{\epsilon}{|A|} \\ a' \neq \arg \max_a Q(s, a) & \text{prob} = \frac{\epsilon}{|A|} \end{cases}$$

최적 수렴을 위한 GLIE 조건

1. 모든 (s, a) 쌍은 무한번 방문되어야 함
2. $\lim_{k \rightarrow \infty} \pi_k(a|s) = 1$

직접 겪으며 학습하는 On-policy: MC와 SARSA

가치 함수를 업데이트하는 대표적인 방법으로는 몬테카를로(MC)와 SARSA가 있음.

SARSA는 에피소드가 끝나길 기다리지 않고 한 스텝만 진행한 뒤 다음 상태의 가치를 추정
에 활용하는 TD(Temporal Difference) 방식을 사용

참고. TD = SARSA 아닌가?

⇒ TD 학습 방식을 행동가치함수(Q함수)와 결합. TD의 한 종류이다.

TD 학습 RECAP:



"에피소드가 끝날 때까지 기다리지 않고 1-step(혹은 n-step)만 간 뒤 다음 상태의 가치를 활용해(Bootstrapping) 현재 가치를 업데이트하는 학습 방식"

Step 1. ϵ -greedy로 행동 선택 → 데이터 생성

$$a_t \sim \pi_\epsilon(s_t), a_{t+1} \sim \pi_\epsilon(s_{t+1})$$

ϵ -greedy는 $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ 경험 튜플을 만드는 역할

Step 2. 그 데이터로 TD update $\rightarrow Q^\pi$ 추정

$$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \underbrace{\alpha(r_t + \gamma Q(s_{t+1}, a_{t+1}))}_{\text{target}}$$

```
init: Q(s,a)=0, N(s,a)=0, ε=1, k=1, π_k = ε-greedy

loop:
    정책 π_k 고정, 에피소드 (s,a,r,...) 샘플링
    observe (r_t, s_{t+1})

    loop (each step):
        a_{t+1} ~ π(s_{t+1})           # 5. take action
        observe (r_{t+1}, s_{t+2})    # 6. observe

        # 7. policy evaluation
        Q(s_t, a_t) ← (1-α)Q(s_t, a_t) + α(r_t + γQ(s_{t+1},
a_{t+1}))

        # 8. policy improvement
        π(s_t) = argmax_a Q(s_t, a) with prob 1-ε, else random

    t = t+1

    k++, ε = 1/k
    π_k ← ε-greedy(Q)
end loop
```

SARSA 수렴 조건

1. 정책이 GLIE 조건을 만족해야 함
2. 학습률 α 에 대한 Robbins-Monro 조건

$$\sum \alpha_t = \infty \quad \sum \alpha_t^2 < \infty \quad \left(\text{예: } \alpha_t = \frac{1}{t} \right)$$

참고) 왜 하필이면 이 조건인가?

1. 학습률의 합이 무한해야 한다 → 어떤 초기값에서 시작해도 결국 도달할 수 있어야 함.

합이 유한하면 업데이트가 너무 일찍 멈춰서 초기 오차를 끝까지 못 지움.

2. 제곱합이 유한해야 한다 → 노이즈가 누적되지 않아야 함.

TD update를 풀어보면:

$$Q(s, a) \leftarrow Q(s, a) + \alpha_t \underbrace{[r + \gamma Q(s', a') - Q(s, a)]}_{\text{true gradient} + \text{noise}}$$

target 안에는 **노이즈**(샘플링 오차)가 섞여 있다. 이 노이즈가 매 스텝 누적될 때 그 크기는:

$$\text{누적 노이즈 분산} \propto \sum_t \alpha_t^2$$

이 합이 발산하면 → 노이즈가 무한히 쌓여서 수렴 불가.

이 합이 수렴하면 → 노이즈가 점점 억제되어 안정적 수렴.

Q-Learning — Off-Policy TD Control

Key idea: Q 추정에서 다음 스텝 최댓값을 bootstrap하여 가져온다.

Bellman Equation 연결

$$\sum_{s'} P(s'|s, a) V^*(s') = \sum_{s'} P(s'|s, a) \max_{a'} Q(s', a') \approx \max_{a'} Q(s_{t+1}, a')$$

모델 없이 실제 행동으로 s_{t+1} 을 얻고, 이를 추정에 활용.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \underbrace{\left(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right)}_{\text{target}}$$

SARSA와 달리 ⇒ **Policy Evaluation 부분만** 차이남.

Initialize:

```
Q(s, a) = 0 for all s, a
Q(terminal, ·) = 0
```

for each episode:

```
s ← initial state
```

```

for each step:
    # 1. 행동 선택 (behavior policy:  $\epsilon$ -greedy)
    a  $\leftarrow \epsilon$ -greedy(Q, s)

    # 2. 환경과 상호작용
    r, s'  $\leftarrow$  env.step(a)

    # 3. TD update (off-policy: max 사용)
    Q(s, a)  $\leftarrow$  Q(s, a) +  $\alpha$  * (r +  $\gamma$  * max_a' Q(s', a') - Q(s, a))

    # 4. 상태 전이
    s  $\leftarrow$  s'

    if s is terminal: break

```

- GLIE 없이 탐욕 탐험만 보장되면 수렴.

비교

SARSA는 (s, a, r, s', a') — 5개 튜플 → 이름이 SARSA

Q-Learning은 (s, a, r, s') — 4개 튜플로 충분

SARSA:

```

a'  $\leftarrow \epsilon$ -greedy(Q, s') # 다음 행동을 미리 샘플링
Q(s, a)  $\leftarrow$  Q(s, a) +  $\alpha$ (r +  $\gamma$ Q(s', a') - Q(s, a)) # 그 행동을 target에 사용

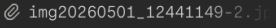
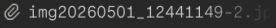
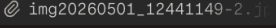
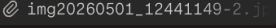
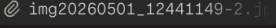
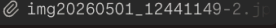
```

Q-Learning:

```

# 다음 행동 샘플링 없음
Q(s, a)  $\leftarrow$  Q(s, a) +  $\alpha$ (r +  $\gamma$  max_a' Q(s', a') - Q(s, a)) # max로 직접 계산

```

학습 방법	장점	단점
MC Control	편향(Unbiased)이 없음, 구현이 단순함 	분산이 높음, 종료가 없는(Continuing) 태스크에 적용 불가 
SARSA	보수적이고 안전한 경로 학습 	행동 탐험이 추정치를 왜곡할 수 있음 (나쁜 행동의 패널티가 과하게 반영됨) 
Q-learning	과감한 최적 경로 학습, 데이터 효율이 높음 	Maximization Bias 문제 (추정치에 max를 취해 실제 참값보다 커지는 편향 발생) 

Maximization Bias:

$\max Q$ 를 취하면 실제 max보다 거짐 → 잘못된 편향 학습

Function Approximation

(s,a) 조합이 너무 많으니, Neural Network로 근사:

현실의 복잡한 문제에서는 (S,A,R)의 조합이 너무 많아 Q값을 표(Tabular) 형태로 모두 저장할 수 없습니다.

그래서 가중치 w 를 가진 신경망을 활용해 Q값을 근사(Function Approximation)하는 방식을 도입.

$$J(w) = E_{\pi} \left[\left(Q^{\pi}(s, a) - \hat{Q}(s, a; w) \right)^2 \right]$$

$$\nabla J(w) = -2E_{\pi} \left[Q^{\pi}(s, a) - \hat{Q}(s, a; w) \right] \nabla_w \hat{Q}(s, a; w)$$

$$\Delta w = \alpha \left[\text{target} - \hat{Q}(s, a; w) \right] \nabla_w \hat{Q}(s, a; w)$$

로 일반화 한다면...

방법	Target
Monte Carlo	G_t
SARSA	$R_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1}; w)$
Q-Learning	$R_{t+1} + \gamma \max_{a'} \hat{Q}(s_{t+1}, a'; w)$

Deadly Triad — DQN의 불안정성 원인

Bootstrapping, Function Approximation, Off-policy Learning

세 요소가 **동시에** 존재할 때 수렴 안 할 수 있음:

원인

1. **Bootstrapping** — 실제 G_t 대신 현재 불완전한 추정치 사용
2. **Function Approximation** — 파라미터를 공유하므로 한 상태 업데이트가 다른 상태에도 영향
3. **Off-policy Learning** — 실제 방문하지 않는 상태의 분포를 기준으로 학습 → 제한할 ground truth 적음

Q-learning w/ Function Approximation

문제점

- **문제점 1 (Non-iid 샘플):** 에이전트가 얻는 순차적 데이터는 시간적 상관관계가 매우 높아 iid 전제를 위반합니다. 신경망 학습의 기본 전제는 iid인데....
- **문제점 2 (Non-stationary target):** 신경망 파라미터가 공유되므로 가중치를 업데이트할 때마다 정답지 역할을 해야 할 타겟 값도 매번 요동칩니다.

$$\text{target} = R_{t+1} + \gamma \max_{a'} \hat{Q}(s_{t+1}, a'; \underbrace{w}_{\text{지금 학습 중인 파라미터}})$$

현재 $w = w_0$ 일 때:

$Q(s, a; w_0) = 5.0$ ← 예측값

$\text{target} = r + \gamma \max Q(s', a'; w_0) = 7.0$

```

w를 7.0 방향으로 업데이트 → w = w_1
Q(s, a; w_1) = 6.2 ← 예측값 올라감
target = r + γ max Q(s', a'; w_1) = 7.8 ← target도 같이 올라가버림

w를 7.8 방향으로 업데이트 → w = w_2
target = 8.3 ← 또 올라감
...

```

DQN 해결책

① Replay Buffer

- $(s, a, r, s') \sim D \rightarrow$ 과거의 미니배치 샘플링
- 시간적 상관관계를 무시하여 학습 안정화
- 같은 경험 데이터를 여러 번 재사용

② Fixed Q-Targets

- Main network $w \rightarrow$ 매 스텝마다 업데이트되는 학습용 네트워크
- Target network $w^- \rightarrow$ 타겟 계산 전용, 가중치 고정, 주기적 업데이트

$$\text{Target} = R_{t+1} + \gamma \max_{a'} \hat{Q}(s_{t+1}, a'; w^-)$$